

Lab_Assignment_3

Each group shall develop **one small application with given functionalities and 3 vulnerabilities** (console-based or Java GUI) based on the application assigned according to the **(group number%10)**. **The objective is not to develop a complete commercial website or mobile application.** Instead, you should implement only the **core functionalities** required to demonstrate the concepts. **Select the language of development as per the SAST tool you installed.**

S. NO.	Application	Five Core Functionalities	Suggested Vulnerabilities (any 3)
1	ATM System	Simulates ATM operations such as login, balance inquiry, withdrawal, deposit, and PIN change	Hardcoded credentials, Improper input validation, SQL Injection, Missing authentication checks, Information leakage through error messages
2	Online Banking	Allows users to transfer funds, check balances, and manage beneficiaries	SQL Injection, Broken Access Control, Insecure Session Management, Insufficient input validation, Sensitive data exposure in logs
3	Hospital Management System	Manages patient registration, appointments, prescriptions, billing, and medical records	SQL Injection, Broken Access Control, File Upload Vulnerability, Missing authorization, Path Traversal
4	Library Management System	Handles book issue/return, member registration, fine calculation, and search operations	SQL Injection, Cross-Site Scripting (XSS), Improper Input Validation, Directory Traversal, Missing authentication
5	Drone Control System	Simulates drone login, waypoint upload, mission execution, telemetry display, and log storage	Command Injection, Missing Authentication, Buffer Overflow (C), Improper Input Validation, Insecure File Handling
6	Cryptocurrency Wallet	Manages wallet creation, transaction history, balance inquiry, and transaction requests (without implementing cryptography)	Broken Access Control, Insecure Session Handling, Input Validation Errors, Hardcoded Secrets, Information Leakage
7	Student Portal	Allows students to log in, register courses, view grades, and update profiles	SQL Injection, Cross-Site Scripting (XSS), Broken Access Control, Insecure Direct Object Reference (IDOR), Missing Authorization
8	E-Commerce Website	Supports product browsing, shopping cart, checkout, and order history	SQL Injection, Cross-Site Scripting (XSS), IDOR, File Upload Vulnerability, Price Manipulation due to poor validation
9	Password Manager	Stores usernames, website names, and passwords (store them in plaintext for the assignment—cryptography will be introduced later)	Hardcoded Credentials, Missing Authentication, Information Leakage, Improper File Permissions, Insecure Storage
10	IoT Device Management	Simulates smart device registration, status monitoring, firmware upload, and configuration management	Command Injection, Path Traversal, Missing Authentication, Insecure File Upload, Improper Input Validation

Lab_Assignment_3

Instructions

1. Continue using the **same GitHub repository (CryptoLabX)** created in the previous laboratory exercise.
2. Inside the repository, create a new folder named:

`secure_application/`

3. This folder will contain the source code, documentation, reports, and all the work related to your assigned application.
4. **Do not create a new repository or change the assigned application** unless instructed.
5. Your repository should have the following structure (Tentative):

```
CryptoLabX/
├── classical/
├── modern/
├── hashing/
├── attacks/
├── analysis/
├── docs/
├── secure_application/
│   ├── src/
│   ├── reports/
│   ├── screenshots/
│   ├── sast/
│   ├── outputs/
│   ├── testcases/
│   └── README.md
└── README.md
```